

```
// =====  
// WEATHER GATE v4.6 - FINAL (NO BATTERY LOGIC)  
// - Удалена вся логика АКБ  
// - Сохранение раз в час  
// - Питание всегда "Сеть"  
// - Только приём WS1080 + BMP280 + RTC  
// =====
```

```
#define MQTT_MAX_PACKET_SIZE 1024
```

```
#include <ETH.h>  
#include <WiFi.h>  
#include <Wire.h>  
#include <RTCLib.h>  
#include <SPI.h>  
#include <LittleFS.h>  
#include <ArduinoJson.h>  
#include <WebServer.h>  
#include <PubSubClient.h>  
#include <Preferences.h>  
#include <HTTPClient.h>  
#include <HTTPUpdate.h>  
#include <esp_task_wdt.h>  
#include <time.h>  
#include <math.h>  
#include <rtl_433_ESP.h>  
#include <RadioLib.h>  
#include <Adafruit_Sensor.h>  
#include <Adafruit_BMP280.h>
```

```
extern CC1101 radio;
```

```
// ===== КОНФИГУРАЦИЯ =====  
SET_LOOP_TASK_STACK_SIZE(20 * 1024);  
#define CURRENT_VERSION "4.6"  
#define FW_NAME "weather_gate"
```

```
// Часовой пояс (UTC+7 — Новосибирск)  
#define TIMEZONE_OFFSET 7
```

```
// Пины WT32-ETH01  
#define I2C_SDA 32  
#define I2C_SCL 33  
#define LED_RF_RX 25
```

```

#define LED_MQTT_TX 26
#define PIN_BAT_ADC 36
#define CC1101_GDO0 4

// Настройки
#define POINTS_HISTORY 288
#define HISTORY_INTERVAL 300000 // 5 минут
#define SENSOR_INTERVAL 60000 // 1 минута
#define MQTT_PUBLISH_INTERVAL 60000 // 1 минута
#define MQTT_RECONNECT_INTERVAL 10000
#define SAVE_INTERVAL 3600000 // 1 час (вместо smartSave)
#define HA_PORT 8123
#define HA_OTA_PATH "/local/ota/weather_gate/"
#define MQTT_TOPIC_WEATHER_DEFAULT "weather_gate/data"
#define BMP280_ADDRESS 0x76
#define BMP_READ_INTERVAL 30000

// Параметры сканера
const int SCAN_STEPS = 21;
const float SCAN_START_FREQ = 914.80;
const float SCAN_STEP = 0.02;
const unsigned long SCAN_STEP_TIME = 65000;

// Усреднение ветра
const int WIND_AVG_SAMPLES = 10;
const int WIND_SPEED_SAMPLES = 10;

// Zambretti
const int PRESSURE_HISTORY_SIZE = 12;

// ===== ЛОГИРОВАНИЕ =====
#define LOG(tag, msg, ...) Serial.printf("[%08lu] [%s] " msg "\n", millis(), tag, ##__VA_ARGS__)
#define LOG_ONCE(tag, msg, ...) do { \
static unsigned long last_log = 0; \
if (millis() - last_log > 30000) { \
Serial.printf("[%08lu] [%s] " msg "\n", millis(), tag, ##__VA_ARGS__); \
last_log = millis(); \
} \
} while(0)

// ===== СТРУКТУРЫ =====
struct RollingHistory {
float temp[POINTS_HISTORY];
float pressure[POINTS_HISTORY];

```

```
float wind[POINTS_HISTORY];
float rain[POINTS_HISTORY];
int head = 0;
} history;
```

```
struct UpdateInfo {
String version;
String fw_url;
String fs_url;
String fw_md5;
String fs_md5;
String changelog;
};
```

```
// ===== ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ =====
```

```
RTC_DS3231 rtc;
Adafruit_BMP280 bmp;
```

```
bool rtc_found = false;
bool bmp_found = false;
bool discovery_sent = false;
```

```
float temperature = 0, humidity = 0, pressure = 1013.25;
float wind_speed_raw = 0, wind_gust_raw = 0;
int wind_direction_raw = 0;
float rain = 0;
float last_rssi = -120.0;
```

```
float temp_min = 0, temp_max = 0;
float pressure_min = 1013.25, pressure_max = 1013.25;
float wind_max = 0, rain_max = 0;
int sensor_id = -1;
String outdoor_battery = "OK";
```

```
// Усреднение ветра
```

```
float wind_dir_history_sin[WIND_AVG_SAMPLES];
float wind_dir_history_cos[WIND_AVG_SAMPLES];
int wind_sample_idx = 0;
bool wind_history_filled = false;
int wind_dir_avg = 0;
```

```
float wind_speed_history[WIND_SPEED_SAMPLES];
int wind_speed_idx = 0;
bool wind_speed_filled = false;
```

```
float wind_avg_smooth = 0;
float wind_gust_max = 0;

// Настройки
float battery_voltage = 0; // Только для информации (не используется)
float current_freq = 915.00;
bool verbose_mode = false;

String hostname = "weather_gate";
String mqtt_ip = "";
String mqtt_user = "";
String mqtt_password = "";
String mqtt_topic_weather = MQTT_TOPIC_WEATHER_DEFAULT;

// BMP280 настройки
float bmp_calibration_offset = 31.7;
int altitude_meters = 150;

// Счётчики и таймеры
uint32_t radio_errors = 0, radio_success = 0;
uint32_t lastSaveTime = 0, lastHistoryTime = 0;
uint32_t lastMQTTReconnect = 0, lastMQTTPublish = 0;
uint32_t last_packet_time = 0;

// Переменные BMP280
float bmp_pressure_raw = 1013.25;
float bmp_temperature = 0;
unsigned long lastBMPRead = 0;

// Сканер
bool scan_active = false;
float scan_results[SCAN_STEPS];
int scan_current_step = 0;
float scan_best_freq = 915.00;
float scan_best_rssi = -150.0;
float scan_step_max_rssi = -150.0;
unsigned long scan_step_start = 0;
float scan_instant_rssi = -120.0;

// Zambretti
float pressure_history_arr[PRESSURE_HISTORY_SIZE];
int pressure_history_idx = 0;
bool pressure_history_full = false;
String zambretti_forecast = "Ожидание данных...";
```

```
String pressure_trend_desc = "Стабильно";  
unsigned long lastZambrettiUpdate = 0;
```

```
// Объекты
```

```
WebServer server(80);  
Preferences pref;  
WiFiClient ethClient;  
PubSubClient mqtt(ethClient);  
rtl_433_ESP rf;  
char msgBuffer[512];
```

```
// ===== ПРОТОТИПЫ =====
```

```
void loadSettings();  
void saveAll();  
void saveCounters();  
void sendDiscovery(const char* name, const char* unit, const char* dev_cla, const char* icon = nullptr);  
void registerAllSensors();  
void publishMQTTData();  
void readSensors();  
void mqttCallback(char* topic, byte* payload, unsigned int length);  
void reconnectMQTT();  
void addHistory(float t, float p, float w, float r);  
void periodicSave();  
void printResetReason();  
String formatDate(DateTime dt);  
float parseSafeFloat(String val);  
bool handleFileRead(String path);  
String getWindDirectionText(int degrees);  
void updateWindAverage(int degrees);  
void updateWindSpeedAverage(float current_speed, float current_gust);  
void updateAutoScanner();  
void updatePressureHistory(float currentPressure);  
float calculatePressureTrend();  
void calculateZambrettiForecast();  
void readBMP280();  
bool getUpdateInfo(UpdateInfo &info);  
int compareVersions(String newVer, String curVer);  
void performOTA();
```

```
// ===== ДИАГНОСТИКА =====
```

```
void printResetReason() {  
    esp_reset_reason_t reason = esp_reset_reason();  
    LOG("SYS", "=== RESET DIAGNOSTIC ===");  
    switch(reason) {
```

```

case ESP_RST_POWERON: LOG("SYS", "Reset: POWER_ON"); break;
case ESP_RST_EXT: LOG("SYS", "Reset: EXTERNAL"); break;
case ESP_RST_SW: LOG("SYS", "Reset: SOFTWARE"); break;
case ESP_RST_PANIC: LOG("SYS", "Reset: PANIC"); break;
case ESP_RST_INT_WDT: LOG("SYS", "Reset: INT WDT"); break;
case ESP_RST_TASK_WDT: LOG("SYS", "Reset: TASK WDT"); break;
case ESP_RST_BROWNOUT: LOG("SYS", "Reset: BROWNOUT"); break;
default: LOG("SYS", "Reset: UNKNOWN (%d)", reason); break;
}
LOG("SYS", "Free heap: %u bytes", ESP.getFreeHeap());
LOG("SYS", "=====");
}

```

// ===== ФОРМАТИРОВАНИЕ ДАТЫ =====

```

String formatDateTime(DateTime dt) {
if (!rtc_found) return "--.--.-- --:--:--";
time_t utc = dt.unixtime();
time_t local = utc + TIMEZONE_OFFSET * 3600;
struct tm *timeinfo = localtime(&local);
char buffer[20];
strftime(buffer, sizeof(buffer), "%d.%m.%Y %H:%M:%S", timeinfo);
return String(buffer);
}

```

// ===== ВСПОМОГАТЕЛЬНЫЕ =====

```

float parseSafeFloat(String val) {
val.replace(',', '.');
val.trim();
return val.toFloat();
}

```

// ===== НАПРАВЛЕНИЕ ВЕТРА =====

```

String getWindDirectionText(int degrees) {
const char* directions[] = {"C", "CCB", "CB", "BCB", "B", "БЮБ", "ЮБ", "ЮЮБ",
"Ю", "ЮЮЗ", "ЮЗ", "ЗЮЗ", "З", "ЗСЗ", "СЗ", "ССЗ"};
int idx = ((int)((degrees + 11) / 22.5)) % 16;
return String(directions[idx]);
}

```

```

void updateWindAverage(int degrees) {
float rad = degrees * M_PI / 180.0;
wind_dir_history_sin[wind_sample_idx] = sin(rad);
wind_dir_history_cos[wind_sample_idx] = cos(rad);
wind_sample_idx++;
}

```

```

if (wind_sample_idx >= WIND_AVG_SAMPLES) {
    wind_sample_idx = 0;
    wind_history_filled = true;
}
int count = wind_history_filled ? WIND_AVG_SAMPLES : wind_sample_idx;
float avg_sin = 0, avg_cos = 0;
for (int i = 0; i < count; i++) {
    avg_sin += wind_dir_history_sin[i];
    avg_cos += wind_dir_history_cos[i];
}
avg_sin /= count;
avg_cos /= count;
float avg_rad = atan2(avg_sin, avg_cos);
wind_dir_avg = (int)(avg_rad * 180.0 / M_PI);
if (wind_dir_avg < 0) wind_dir_avg += 360;
}

void updateWindSpeedAverage(float current_speed, float current_gust) {
    wind_speed_history[wind_speed_idx] = current_speed;
    wind_speed_idx++;
    if (wind_speed_idx >= WIND_SPEED_SAMPLES) {
        wind_speed_idx = 0;
        wind_speed_filled = true;
    }
    float sum = 0;
    int count = wind_speed_filled ? WIND_SPEED_SAMPLES : wind_speed_idx;
    for (int i = 0; i < count; i++) sum += wind_speed_history[i];
    wind_avg_smooth = sum / count;
    wind_gust_max = current_gust;
}

// ===== BMP280 =====
void readBMP280() {
    if (!bmp_found) return;
    float p_raw = bmp.readPressure() / 100.0F; // Давление в hPa
    // Фильтр "мусора" (защита от подвисания шины I2C)
    if (p_raw < 850.0 || p_raw > 1100.0) {
        LOG("BMP280", "Invalid reading ignored (%.1f hPa)", p_raw);
        return;
    }
    bmp_pressure_raw = p_raw;
    bmp_temperature = bmp.readTemperature();
    // ПРАВИЛЬНЫЙ РАСЧЕТ: Приведение к уровню моря (Sea Level Pressure)
    // Используем встроенную функцию библиотеки, учитывающую высоту

```

```

float calibrated = bmp_pressure_raw + bmp_calibration_offset;
pressure = bmp.seaLevelForAltitude(altitude_meters, calibrated);
LOG_ONCE("BMP280", "Raw=%.1f, MSL=%.1f, T=%.1f", bmp_pressure_raw, pressure, bmp_temperature);
}

```

```

// ===== ZAMBRETTI =====

```

```

void updatePressureHistory(float currentPressure) {
    pressure_history_arr[pressure_history_idx] = currentPressure;
    pressure_history_idx = (pressure_history_idx + 1) % PRESSURE_HISTORY_SIZE;
    if (pressure_history_idx == 0) pressure_history_full = true;
}

```

```

float calculatePressureTrend() {
    if (!pressure_history_full) return 0.0;
    int firstIdx = pressure_history_idx;
    int lastIdx = (pressure_history_idx - 1 + PRESSURE_HISTORY_SIZE) % PRESSURE_HISTORY_SIZE;
    return pressure_history_arr[lastIdx] - pressure_history_arr[firstIdx];
}

```

```

void calculateZambrettiForecast() {
    float trend = calculatePressureTrend();
    if (trend > 1.5) pressure_trend_desc = "Резкий подъём ↑ ↑ ↑";
    else if (trend > 0.5) pressure_trend_desc = "Подъём ↑ ↑";
    else if (trend < -1.5) pressure_trend_desc = "Резкий спад ↓ ↓ ↓";
    else if (trend < -0.5) pressure_trend_desc = "Спад ↓ ↓";
    else pressure_trend_desc = "Стабильно →";
    float p = pressure;
    if (p > 1030.0 && trend > 0.0) zambretti_forecast = "Отличная, ясная погода ☀️";
    else if (p > 1025.0) zambretti_forecast = "Ясная погода ☀️";
    else if (p > 1020.0 && trend < -0.5) zambretti_forecast = "Переменная облачность 🌤️";
    else if (p > 1020.0) zambretti_forecast = "Хорошая погода ☀️";
    else if (p > 1015.0 && trend > 0.0) zambretti_forecast = "Улучшение погоды ☀️";
    else if (p > 1015.0) zambretti_forecast = "Облачно с прояснениями 🌤️";
    else if (p > 1010.0 && trend < -0.5) zambretti_forecast = "Вероятны осадки 🌧️";
    else if (p > 1010.0) zambretti_forecast = "Облачно ☁️";
    else if (p > 1005.0 && trend < -0.5) zambretti_forecast = "Дождливая погода 🌧️";
    else if (p > 1005.0) zambretti_forecast = "Пасмурно ☁️";
    else if (p > 1000.0) zambretti_forecast = "Дожди 🌧️";
    else zambretti_forecast = "Штормовое предупреждение ⚡️";
    LOG_ONCE("ZAMBRETTI", "%s (MSL=%.1f hPa, trend=%.1f)",
    zambretti_forecast.c_str(), p, trend);
}

```

```

// ===== СКАНЕР =====

```



```

void updateAutoScanner() {
    if (!scan_active) return;
    if (scan_step_start == 0) {
        scan_step_start = millis();
        float freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
        radio.setFrequency(freq);
        radio.setRxBandwidth(100.0);
        radio.setFrequencyDeviation(50.0);
        radio.setBitRate(9.6);
        radio.startReceive();
        delay(20);
        scan_step_max_rssi = radio.getRSSI();
        scan_instant_rssi = scan_step_max_rssi;
        LOG("SCAN", "Step %d: %.3f MHz", scan_current_step, freq);
        return;
    }
    if (millis() - scan_step_start > SCAN_STEP_TIME) {
        scan_results[scan_current_step] = scan_step_max_rssi;
        if (scan_step_max_rssi > scan_best_rssi) {
            scan_best_rssi = scan_step_max_rssi;
            scan_best_freq = SCAN_START_FREQ + (scan_current_step * SCAN_STEP);
            LOG("SCAN", ">>> Best: %.3f MHz (%.1f dBm)", scan_best_freq, scan_best_rssi);
        }
        scan_current_step++;
        scan_step_start = 0;
        if (scan_current_step >= SCAN_STEPS) {
            scan_active = false;
            rf.initReceiver(CC1101_GDO0, current_freq);
            rf.enableReceiver();
            if (scan_best_rssi > -95.0) {
                current_freq = scan_best_freq;
                radio.setFrequency(current_freq);
                saveAll();
                LOG("SCAN", "OPTIMAL FREQUENCY: %.3f MHz", current_freq);
            }
        } else {
            delay(10);
        }
    }
}

// ===== RTL_433 CALLBACK =====
void rtl_433_Callback(char* message) {
    digitalWrite(LED_RF_RX, HIGH);
}

```

```

last_rssi = radio.getRSSI();
StaticJsonDocument<1024> doc;
DeserializationError error = deserializeJson(doc, message);
if (!error) {
    radio_success++;
    last_packet_time = millis();
    // Извлекаем данные (оставляем вашу логику парсинга)
    if (doc.containsKey("id")) sensor_id = doc["id"];
    if (doc.containsKey("temperature_C")) temperature = doc["temperature_C"];
    else if (doc.containsKey("temp_C")) temperature = doc["temp_C"];
    if (doc.containsKey("humidity")) humidity = doc["humidity"];
    if (doc.containsKey("wind_avg_m_s")) wind_speed_raw = doc["wind_avg_m_s"];
    if (doc.containsKey("wind_gust_m_s")) wind_gust_raw = doc["wind_gust_m_s"];
    if (doc.containsKey("wind_dir_deg")) wind_direction_raw = doc["wind_dir_deg"];
    if (doc.containsKey("rain_mm")) rain = doc["rain_mm"];
    // Обновляем усреднения
    if (doc.containsKey("wind_avg_m_s")) updateWindSpeedAverage(wind_speed_raw, wind_gust_raw);
    if (doc.containsKey("wind_dir_deg")) updateWindAverage(wind_direction_raw);
    if (doc.containsKey("battery_ok")) {
        outdoor_battery = (doc["battery_ok"] == 1) ? "OK" : "LOW";
    }

    // Логируем в Serial (без тяжелых функций)
    LOG("WS1080", "New packet received. RSSI: %.1f", last_rssi);
    // ВАЖНО: Мы НЕ вызываем здесь publishMQTTData().
    // Данные уйдут в основном цикле loop() по таймеру.
} else {
    radio_errors++;
}
digitalWrite(LED_RF_RX, LOW);
}

// ===== MQTT =====
void mqttCallback(char* topic, byte* payload, unsigned int length) {}

void sendDiscovery(const char* name, const char* unit, const char* dev_cla, const char* icon) {
    if (!mqtt.connected()) return;
    String deviceId = hostname;
    String topic = "homeassistant/sensor/" + deviceId + "/" + name + "/config";
    StaticJsonDocument<512> doc;
    doc["name"] = deviceId + " " + name;
    doc["state_topic"] = ("smart/" + deviceId + "/" + name + "/state").c_str();
    doc["value_template"] = ("{{ value_json." + String(name) + " }}").c_str();
    doc["unique_id"] = (deviceId + "_" + name).c_str();
}

```

```

if (unit && strlen(unit) > 0) doc["unit_of_measurement"] = unit;
if (dev_cla && strlen(dev_cla) > 0) doc["device_class"] = dev_cla;
if (icon) doc["icon"] = icon;
JsonObject device = doc.createNestedObject("device");
device["identifiers"] = deviceId;
device["name"] = "Weather Gate";
device["sw_version"] = CURRENT_VERSION;
device["model"] = "WS1080 Gateway + BMP280";
device["manufacturer"] = "DIY";
String out;
serializeJson(doc, out);
mqtt.publish(topic.c_str(), out.c_str(), true);
}

```

```

void registerAllSensors() {
sendDiscovery("temperature", "°C", "temperature", "mdi:thermometer");
sendDiscovery("humidity", "%", "humidity", "mdi:water-percent");
sendDiscovery("pressure", "hPa", "pressure", "mdi:gauge");
sendDiscovery("pressure_mmhg", "mmHg", "pressure", "mdi:gauge");
sendDiscovery("wind_speed", "m/s", "wind_speed", "mdi:weather-windy");
sendDiscovery("wind_gust", "m/s", "wind_speed", "mdi:weather-windy-variant");
sendDiscovery("wind_direction", "°", nullptr, "mdi:compass");
sendDiscovery("rain", "mm", "precipitation", "mdi:weather-rainy");
sendDiscovery("outdoor_battery", nullptr, "battery", "mdi:battery");
sendDiscovery("forecast", nullptr, nullptr, "mdi:weather-partly-cloudy");
sendDiscovery("rssi", "dBm", "signal_strength", "mdi:signal");
sendDiscovery("freq", "MHz", nullptr, "mdi:radio-tower");
sendDiscovery("bmp_temperature", "°C", "temperature", "mdi:thermometer");
sendDiscovery("bmp_pressure", "hPa", "pressure", "mdi:gauge");
sendDiscovery("bmp_pressure_mmhg", "mmHg", "pressure", "mdi:gauge");
LOG("MQTT", "All sensors registered");
}

```

```

void publishMQTTData() {
if (!mqtt.connected()) return;
StaticJsonDocument<1024> doc;
doc["temperature"] = temperature;
doc["humidity"] = humidity;
doc["pressure"] = pressure;
doc["pressure_mmhg"] = pressure * 0.750062;
doc["wind_speed"] = wind_avg_smooth;
doc["wind_gust"] = wind_gust_max;
doc["wind_direction"] = wind_dir_avg;
doc["rain"] = rain;

```

```

doc["outdoor_battery"] = outdoor_battery;
doc["sensor_id"] = sensor_id;
doc["forecast"] = zambretti_forecast;
doc["rssi"] = last_rssi;
doc["freq"] = current_freq;
doc["bmp_temperature"] = bmp_temperature;
doc["bmp_pressure"] = bmp_pressure_raw + bmp_calibration_offset;
doc["bmp_pressure_mmhg"] = (bmp_pressure_raw + bmp_calibration_offset) * 0.750062;
doc["radio_errors"] = radio_errors;
doc["radio_success"] = radio_success;
doc["power_source"] = "AC";
doc["uptime"] = millis() / 1000;
if (rtc_found) {
doc["datetime"] = formatDateTime(rtc.now());
}
String out;
serializeJson(doc, out);
mqtt.publish(("smart/" + hostname + "/state").c_str(), out.c_str(), false);
}

```

```

void reconnectMQTT() {
if (mqtt_ip.length() < 7) return;
String clientId = "WeatherGate_" + hostname + "_" + String(ESP.getEfuseMac(), HEX);
if (mqtt.connect(clientId.c_str(), mqtt_user.c_str(), mqtt_password.c_str())) {
LOG("MQTT", "Connected to broker");
if (!discovery_sent) {
registerAllSensors();
discovery_sent = true;
}
} else {
LOG("MQTT", "Failed, rc=%d", mqtt.state());
}
}

```

```

// ===== OTA =====
bool getUpdateInfo(UpdateInfo &info) {
if (mqtt_ip.length() < 7) return false;
String url = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH + "version.json";
HTTPClient http;
http.setTimeout(5000);
if (!http.begin(url)) return false;
int httpCode = http.GET();
if (httpCode != HTTP_CODE_OK) {
http.end();
}
}

```

```

return false;
}
String payload = http.getString();
http.end();
StaticJsonDocument<768> doc;
DeserializationError error = deserializeJson(doc, payload);
if (error) return false;
info.version = doc["version"] | "";
info.fw_md5 = doc["fw_md5"] | "";
info.fs_md5 = doc["fs_md5"] | "0";
info.changelog = doc["changelog"] | "";
String baseUrl = "http://" + mqtt_ip + ":" + String(HA_PORT) + HA_OTA_PATH;
info.fw_url = baseUrl + "firmware.bin";
info.fs_url = baseUrl + "littelfs.bin";
return info.version.length() > 0;
}

```

```

int compareVersions(String newVer, String curVer) {
if (newVer == curVer) return 0;
int newMajor = 0, newMinor = 0, newPatch = 0;
int curMajor = 0, curMinor = 0, curPatch = 0;
sscanf(newVer.c_str(), "%d.%d.%d", &newMajor, &newMinor, &newPatch);
sscanf(curVer.c_str(), "%d.%d.%d", &curMajor, &curMinor, &curPatch);
if (newMajor > curMajor) return 1;
if (newMajor < curMajor) return -1;
if (newMinor > curMinor) return 1;
if (newMinor < curMinor) return -1;
if (newPatch > curPatch) return 1;
if (newPatch < curPatch) return -1;
return 0;
}

```

```

void performOTA() {
LOG("OTA", "=== OTA START ===");
UpdateInfo info;
if (!getUpdateInfo(info)) {
LOG("OTA", "Failed to get update info");
return;
}
if (info.version == CURRENT_VERSION) {
LOG("OTA", "Already up to date");
return;
}
LOG("OTA", "Updating to %s", info.version.c_str());

```

```
mqtt.disconnect();
delay(500);
WiFiClient otaClient;
otaClient.setTimeout(15000);
if (info.fs_md5.length() > 2 && info.fs_md5 != "0") {
    LOG("OTA", "Updating LittleFS...");
    httpUpdate.setMD5sum(info.fs_md5);
    httpUpdate.updateFs(otaClient, info.fs_url);
    delay(500);
}
LOG("OTA", "Updating firmware...");
httpUpdate.setMD5sum(info.fw_md5);
httpUpdate.rebootOnUpdate(true);
httpUpdate.update(otaClient, info.fw_url);
}
```

```
// ===== ИСТОРИЯ =====
```

```
void addHistory(float t, float p, float w, float r) {
    history.temp[history.head] = t;
    history.pressure[history.head] = p;
    history.wind[history.head] = w;
    history.rain[history.head] = r;
    history.head = (history.head + 1) % POINTS_HISTORY;
}
```

```
// ===== ПЕРИОДИЧЕСКОЕ СОХРАНЕНИЕ =====
```

```
void periodicSave() {
    if (millis() - lastSaveTime >= SAVE_INTERVAL) {
        saveCounters();
        lastSaveTime = millis();
        LOG("SYS", "Periodic save");
    }
}
```

```
// ===== ДАТЧИКИ =====
```

```
void readSensors() {
    unsigned long now = millis();
    if (now - lastBMPRead > BMP_READ_INTERVAL) {
        lastBMPRead = now;
        readBMP280();
    }
}
```

```
// ===== НАСТРОЙКИ =====
```

```

void loadSettings() {
    pref.begin("weather", true);
    hostname = pref.getString("hostname", "weather_gate");
    if (hostname.length() == 0) hostname = "weather_gate";
    mqtt_ip = pref.getString("mq_ip", "");
    mqtt_user = pref.getString("mq_u", "");
    mqtt_password = pref.getString("mq_p", "");
    mqtt_topic_weather = pref.getString("mqtt_topic_weather", MQTT_TOPIC_WEATHER_DEFAULT);
    current_freq = pref.getFloat("freq", 915.0);
    radio_errors = pref.getUInt("radio_errors", 0);
    radio_success = pref.getUInt("radio_success", 0);
    temperature = pref.getFloat("temp", 0);
    humidity = pref.getFloat("hum", 0);
    pressure = pref.getFloat("pres", 1013.25);
    temp_min = pref.getFloat("temp_min", 0);
    temp_max = pref.getFloat("temp_max", 0);
    pressure_min = pref.getFloat("pres_min", 1013.25);
    pressure_max = pref.getFloat("pres_max", 1013.25);
    wind_max = pref.getFloat("wind_max", 0);
    rain = pref.getFloat("rain", 0);
    rain_max = pref.getFloat("rain_max", 0);
    sensor_id = pref.getInt("sensor_id", -1);
    bmp_calibration_offset = pref.getFloat("bmp_offset", 31.7);
    altitude_meters = pref.getInt("altitude", 150);
    pref.end();
    LOG("SYS", "Settings loaded: freq=%.2f, alt=%d, offset=%.1f",
        current_freq, altitude_meters, bmp_calibration_offset);
}

```

```

void saveAll() {
    pref.begin("weather", false);
    pref.putString("hostname", hostname);
    pref.putString("mq_ip", mqtt_ip);
    pref.putString("mq_u", mqtt_user);
    pref.putString("mq_p", mqtt_password);
    pref.putString("mqtt_topic_weather", mqtt_topic_weather);
    pref.putFloat("freq", current_freq);
    pref.putUInt("radio_errors", radio_errors);
    pref.putUInt("radio_success", radio_success);
    pref.putFloat("temp", temperature);
    pref.putFloat("hum", humidity);
    pref.putFloat("pres", pressure);
    pref.putFloat("rain", rain);
    pref.putFloat("rain_max", rain_max);
}

```

```

pref.putFloat("temp_min", temp_min);
pref.putFloat("temp_max", temp_max);
pref.putFloat("pressure_min", pressure_min);
pref.putFloat("pressure_max", pressure_max);
pref.putFloat("wind_max", wind_max);
pref.putInt("sensor_id", sensor_id);
pref.putFloat("bmp_offset", bmp_calibration_offset);
pref.putInt("altitude", altitude_meters);
pref.end();
LOG("SYS", "All settings saved");
}

```

```

void saveCounters() {
pref.begin("weather", false);
pref.putUInt("radio_errors", radio_errors);
pref.putUInt("radio_success", radio_success);
pref.putFloat("temp", temperature);
pref.putFloat("hum", humidity);
pref.putFloat("pres", pressure);
pref.putFloat("rain", rain);
pref.end();
}

```

```

// ===== GZIP ОБРАБОТЧИК =====
bool handleFileRead(String path) {
if (path.endsWith("/")) path += "index.html";
String contentType = "text/plain";
if (path.endsWith(".html")) contentType = "text/html";
else if (path.endsWith(".js")) contentType = "application/javascript";
else if (path.endsWith(".css")) contentType = "text/css";
else if (path.endsWith(".ico")) contentType = "image/x-icon";
String gzPath = path + ".gz";
if (LittleFS.exists(gzPath)) {
File file = LittleFS.open(gzPath, "r");
if (file) {
server.setContentLength(file.size());
server.setHeader("Content-Encoding", "gzip");
server.setHeader("Cache-Control", "public, max-age=86400");
server.send(200, contentType, "");
server.client().write(file);
file.close();
LOG("FS", "Sent gzipped: %s", gzPath.c_str());
return true;
}
}
}

```



```

}
if (LittleFS.exists(path)) {
  File file = LittleFS.open(path, "r");
  if (file) {
    server.streamFile(file, contentType);
    file.close();
    LOG("FS", "Sent plain: %s", path.c_str());
    return true;
  }
}
return false;
}

// ===== SETUP =====
void setup() {
  Serial.begin(115200);
  delay(500);
  printResetReason();
  pinMode(LED_RF_RX, OUTPUT);
  pinMode(LED_MQTT_TX, OUTPUT);
  digitalWrite(LED_RF_RX, LOW);
  digitalWrite(LED_MQTT_TX, LOW);
  analogSetAttenuation(ADC_11db);
  // Инициализация массивов усреднения
  for (int i = 0; i < WIND_AVG_SAMPLES; i++) {
    wind_dir_history_sin[i] = 0;
    wind_dir_history_cos[i] = 0;
    wind_speed_history[i] = 0;
  }
  for (int i = 0; i < SCAN_STEPS; i++) scan_results[i] = -120.0;
  // ===== I2C =====
  Wire.begin(I2C_SDA, I2C_SCL);
  Wire.setClock(100000);
  delay(200);
  // RTC DS3231
  if (rtc.begin()) {
    rtc_found = true;
    LOG("RTC", "DS3231 found");
    delay(100);
  } else {
    LOG("RTC", "DS3231 NOT found");
  }
  // BMP280
  delay(100);

```

```

if (bmp.begin(BMP280_ADDRESS)) {
  bmp_found = true;
  bmp.setSampling(Adafruit_BMP280::MODE_NORMAL,
  Adafruit_BMP280::SAMPLING_X2,
  Adafruit_BMP280::SAMPLING_X16,
  Adafruit_BMP280::FILTER_X16,
  Adafruit_BMP280::STANDBY_MS_500);
  LOG("BMP280", "Sensor found at 0x%02X", BMP280_ADDRESS);
  readBMP280();
} else {
  LOG("BMP280", "Sensor NOT found");
}
// LittleFS
if (!LittleFS.begin(true)) {
  LOG("FS", "LittleFS mount failed");
} else {
  LOG("FS", "LittleFS mounted");
}
// ===== ETHERNET =====
WiFi.mode(WIFI_OFF);
btStop();
ETH.begin(ETH_PHY_LAN8720, 1, 23, 18, 16, ETH_CLOCK_GPIO17_OUT);
delay(500);
unsigned long startWait = millis();
while (ETH.localIP().toString() == "0.0.0.0" && millis() - startWait < 10000) {
  delay(100);
}
LOG("ETH", "IP: %s", ETH.localIP().toString().c_str());
loadSettings();
if (hostname.length() > 0) ETH.setHostname(hostname.c_str());
// ===== РАДИО =====
LOG("CC1101", "Initializing...");
rf.setCallback(rtl_433_Callback, msgBuffer, 512);
rf.rtlVerbose = 0;
radio.setBitRate(9.6);
radio.setRxBandwidth(100.0);
radio.setFrequencyDeviation(50.0);
rf.initReceiver(CC1101_GDO0, current_freq);
rf.enableReceiver();
LOG("CC1101", "Receiver ready at %.2f MHz", current_freq);
// ===== MQTT =====
if (mqtt_ip.length() > 5) {
  mqtt.setServer(mqtt_ip.c_str(), 1883);
  mqtt.setCallback(mqttCallback);
}

```

```

mqtt.setKeepAlive(60);
LOG("MQTT", "Server set to %s", mqtt_ip.c_str());
}
// ===== ИНИЦИАЛИЗАЦИЯ ИСТОРИИ =====
for (int i = 0; i < POINTS_HISTORY; i++) {
    history.temp[i] = temperature;
    history.pressure[i] = pressure;
    history.wind[i] = 0;
    history.rain[i] = 0;
}
for (int i = 0; i < PRESSURE_HISTORY_SIZE; i++) pressure_history_arr[i] = pressure;
// ===== WEB-СЕРВЕР =====
server.on("/api/data", HTTP_GET, []() {
    StaticJsonDocument<2048> doc;
    doc["temperature"] = temperature;
    doc["humidity"] = humidity;
    doc["pressure"] = pressure;
    doc["pressure_mmhg"] = pressure * 0.750062;
    doc["wind_speed"] = wind_avg_smooth;
    doc["wind_gust"] = wind_gust_max;
    doc["wind_direction"] = wind_dir_avg;
    doc["wind_dir_txt"] = getWindDirectionText(wind_dir_avg);
    doc["rain"] = rain;
    doc["rain_max"] = rain_max;
    doc["temp_min"] = temp_min;
    doc["temp_max"] = temp_max;
    doc["pressure_min"] = pressure_min;
    doc["pressure_max"] = pressure_max;
    doc["wind_max"] = wind_max;
    doc["outdoor_battery"] = outdoor_battery;
    doc["sensor_id"] = sensor_id;
    doc["power_source"] = "AC";
    doc["mqtt_status"] = mqtt.connected() ? "OK" : "Offline";
    doc["hostname"] = hostname;
    doc["version"] = CURRENT_VERSION;
    doc["uptime"] = millis() / 1000;
    doc["radio_errors"] = radio_errors;
    doc["radio_success"] = radio_success;
    doc["current_freq"] = current_freq;
    doc["mqtt_ip"] = mqtt_ip;
    doc["mqtt_user"] = mqtt_user;
    doc["mqtt_topic_weather"] = mqtt_topic_weather;
    doc["forecast"] = zambretti_forecast;
    doc["pressure_trend"] = pressure_trend_desc;

```

```

doc["rssi"] = last_rssi;
doc["rssi_current"] = radio.getRSSI();
doc["bmp_temperature"] = bmp_temperature;
doc["bmp_pressure"] = bmp_pressure_raw + bmp_calibration_offset;
doc["bmp_pressure_mmhg"] = (bmp_pressure_raw + bmp_calibration_offset) * 0.750062;
doc["altitude"] = altitude_meters;
doc["bmp_offset"] = bmp_calibration_offset;
if (rtc_found) doc["datetime"] = formatDate(rtc.now());
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
server.on("/api/history", HTTP_GET, []() {
  StaticJsonDocument<6144> doc;
  JSONArray tempHist = doc.createNestedArray("temp_history");
  JSONArray pressureHist = doc.createNestedArray("pressure_history");
  JSONArray windHist = doc.createNestedArray("wind_history");
  JSONArray rainHist = doc.createNestedArray("rain_history");
  for (int i = 0; i < POINTS_HISTORY; i++) {
    int idx = (history.head + i) % POINTS_HISTORY;
    tempHist.add(history.temp[idx]);
    pressureHist.add(history.pressure[idx]);
    windHist.add(history.wind[idx]);
    rainHist.add(history.rain[idx]);
  }
  String buf;
  serializeJson(doc, buf);
  server.send(200, "application/json", buf);
});
server.on("/api/sync_time", HTTP_POST, []() {
  if (server.hasArg("t") && rtc_found) {
    rtc.adjust(DateTime(server.arg("t").toInt()));
    server.send(200, "text/plain", "OK");
  } else {
    server.send(400, "text/plain", "Failed");
  }
});
server.on("/api/set_freq", HTTP_POST, []() {
  if (server.hasArg("f")) {
    float new_freq = server.arg("f").toFloat();
    if (new_freq >= 860.0 && new_freq <= 930.0) {
      current_freq = new_freq;
      pref.begin("weather", false);
      pref.putFloat("freq", current_freq);
    }
  }
});

```

```

pref.end();
rf.disableReceiver();
delay(50);
rf.initReceiver(CC1101_GDO0, current_freq);
rf.enableReceiver();
LOG("RF", "Frequency changed to %.2f MHz", current_freq);
server.send(200, "text/plain", "OK");
} else {
server.send(400, "text/plain", "Invalid frequency");
}
}
});
server.on("/api/set_verbose", HTTP_POST, []() {
if (server.hasArg("v")) {
verbose_mode = (server.arg("v") == "1");
rf.rtlVerbose = verbose_mode ? 1 : 0;
LOG("SYS", "Verbose mode: %s", verbose_mode ? "ON" : "OFF");
server.send(200, "text/plain", "OK");
}
});
server.on("/api/reset_errors", HTTP_POST, []() {
radio_errors = 0;
radio_success = 0;
saveCounters();
server.send(200, "text/plain", "OK");
});
server.on("/api/check_update", HTTP_GET, []() {
StaticJsonDocument<512> doc;
UpdateInfo info;
if (getUpdateInfo(info)) {
doc["has_update"] = (info.version != CURRENT_VERSION);
doc["current_version"] = CURRENT_VERSION;
doc["new_version"] = info.version;
doc["changelog"] = info.changelog;
} else {
doc["has_update"] = false;
doc["error"] = "Failed to fetch update info";
}
String buf;
serializeJson(doc, buf);
server.send(200, "application/json", buf);
});
server.on("/api/do_update", HTTP_POST, []() {
static unsigned long lastOTA = 0;

```

```

if (millis() - lastOTA < 60000) {
    server.send(429, "text/plain", "Too frequent requests");
    return;
}
lastOTA = millis();
server.send(200, "text/plain", "Update started...");
delay(500);
performOTA();
});
server.on("/api/start_scan", HTTP_GET, []() {
    if (!scan_active) {
        rf.disableReceiver();
        scan_active = true;
        scan_current_step = 0;
        scan_best_rssi = -150.0;
        scan_step_start = 0;
        for (int i = 0; i < SCAN_STEPS; i++) scan_results[i] = -120.0;
        LOG("API", "Spectrum scan started");
        server.send(200, "text/plain", "OK");
    } else {
        server.send(409, "text/plain", "Scan already active");
    }
});
server.on("/api/scan_data", HTTP_GET, []() {
    StaticJsonDocument<1024> doc;
    doc["active"] = scan_active;
    doc["step"] = scan_current_step;
    doc["instant"] = scan_instant_rssi;
    doc["rssi_current"] = radio.getRSSI();
    JsonArray data = doc.createNestedArray("rssi");
    for (int i = 0; i < SCAN_STEPS; i++) data.add(scan_results[i]);
    String buf;
    serializeJson(doc, buf);
    server.send(200, "application/json", buf);
});
server.on("/api/config", HTTP_POST, []() {
    String body = server.arg("plain");
    StaticJsonDocument<2048> doc;
    DeserializationError error = deserializeJson(doc, body);
    if (error) {
        LOG("API", "JSON parse error: %s", error.c_str());
        server.send(400, "text/plain", "JSON Error");
        return;
    }
}

```

```

if (doc.containsKey("hostname")) hostname = doc["hostname"].as<String>();
if (doc.containsKey("mq_ip")) mqtt_ip = doc["mq_ip"].as<String>();
if (doc.containsKey("mq_u")) mqtt_user = doc["mq_u"].as<String>();
if (doc.containsKey("mq_p")) mqtt_password = doc["mq_p"].as<String>();
if (doc.containsKey("mqtt_topic_weather")) mqtt_topic_weather = doc["mqtt_topic_weather"].as<String>();
if (doc.containsKey("altitude")) altitude_meters = doc["altitude"].as<int>();
if (doc.containsKey("bmp_offset")) bmp_calibration_offset =
parseSafeFloat(doc["bmp_offset"].as<String>());
if (hostname.length() > 0) ETH.setHostname(hostname.c_str());
mqtt.setServer(mqtt_ip.c_str(), 1883);
saveAll();
server.send(200, "OK");
LOG("API", "Settings saved, restarting...");
delay(500);
ESP.restart();
});
server.on("/", HTTP_GET, []() {
if (!handleFileRead("/")) {
server.send(404, "text/plain", "index.html not found");
}
});
server.onNotFound([]() {
if (!handleFileRead(server.uri())) {
server.send(404, "text/plain", "Not Found");
}
});
server.begin();
LOG("HTTP", "Server started");
lastSaveTime = millis();
lastHistoryTime = millis();
lastBMPRead = millis();
LOG("SYS", "Setup complete");
LOG("SYS", "Waiting for WS1080 packets at %.2f MHz", current_freq);
}

// ===== LOOP =====
void loop() {
// 1. ПРИОРИТЕТ №1: Радио (вызываем максимально часто)
if (!scan_active) {
rf.loop();
} else {
updateAutoScanner();
}
}

```

```
// 2. Веб-сервер
server.handleClient();

// 3. Опрос датчиков (раз в 1 минуту - SENSOR_INTERVAL)
static uint32_t lastSensors = 0;
if (millis() - lastSensors >= SENSOR_INTERVAL) {
    lastSensors = millis();
    readSensors();
}

// 4. История и Прогноз (раз в 5 минут)
if (!scan_active && (millis() - lastHistoryTime >= HISTORY_INTERVAL)) {
    lastHistoryTime = millis();
    addHistory(temperature, pressure, wind_avg_smooth, rain);
    updatePressureHistory(pressure);
    calculateZambrettiForecast();
}

// 5. MQTT (Публикация и поддержание связи)
if (mqtt.connected()) {
    mqtt.loop();
    if (millis() - lastMQTTPublish >= MQTT_PUBLISH_INTERVAL) {
        lastMQTTPublish = millis();
        publishMQTTData();
    }
} else if (mqtt_ip.length() > 5 && millis() - lastMQTTReconnect >= MQTT_RECONNECT_INTERVAL) {
    lastMQTTReconnect = millis();
    reconnectMQTT();
}

// 6. Сохранение (раз в час)
periodicSave();

// Минимальная пауза для предотвращения срабатывания Watchdog,
// но не мешающая радио.
delay(1);
}
```